

Guangzhou Invitational Competition Test Project

Mobile Applications Development (Skill 08)

Module A Functionality

Cash Escort Record

An armored cash route daily record app.

Mobile-related features in this TP: biometric sign-in and GPS location capture.

Recommended competition time: 2.5 hours

Contents

Cash Escort Record	1
Contents	2
Introduction	3
Description of project and tasks	3
GPS coordinates for testing the current-location feature	3
Reference Wireframes	4
General Demands	5
1. Sign In Page	6
2. Daily Record Page	7
3. Add Cash Stop Page	8
4. Check Out Summary Page	9
5. Records List Page	10
Calculation Rules and Testing Notes	11
Preloaded data files	12
Instructions to the Competitor	12
Marking Scheme	12

Introduction

Cash Escort Record is designed for an armored cash transport team that collects cash from different pickup spots during a day route. The application helps the operator authenticate securely, create a daily record, save each cash-in stop with a location, and complete the route with a final summary.

As a competitor in mobile applications development, your task is to build the application according to the wireframes and the functional requirements below. The solution should support local persistence and should work with mobile features that can be tested in Flutter, Android native, and iOS native solutions.

Description of project and tasks

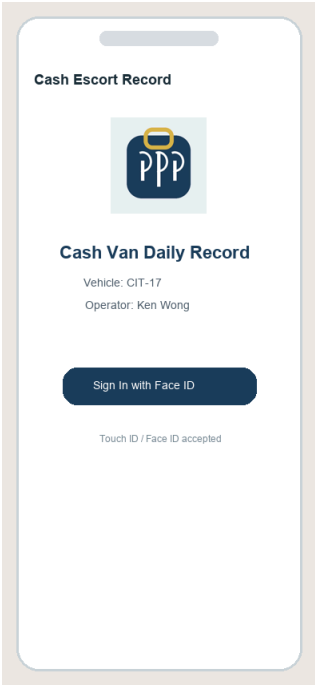
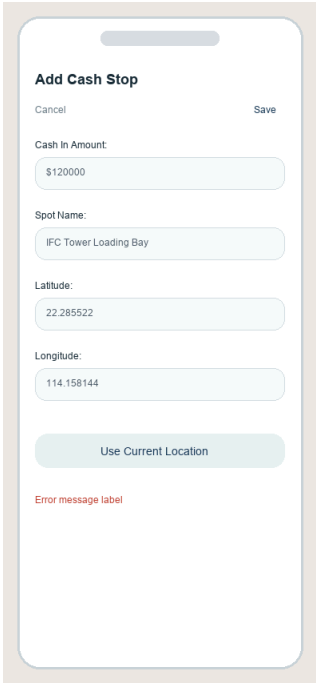
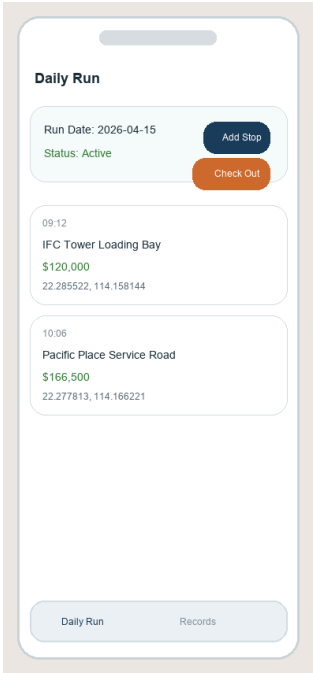
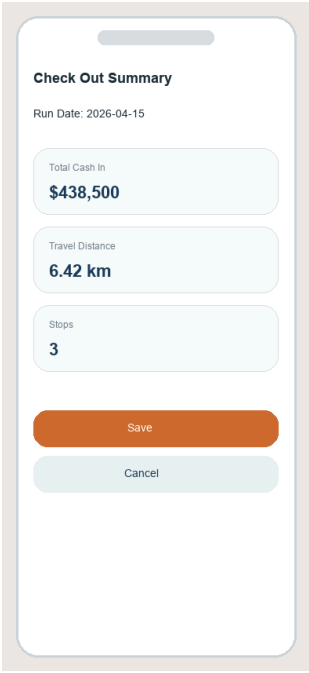
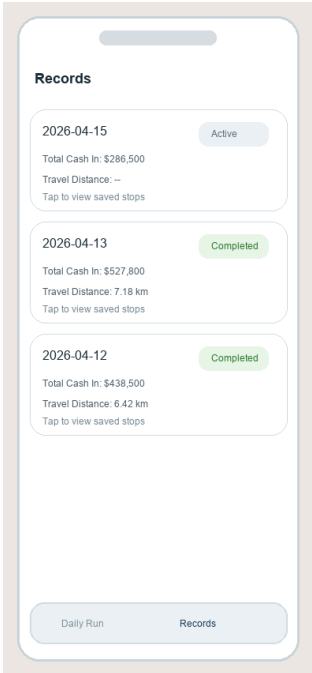
Below is an overview of the module format.

Module Name	Competition Time	Marking	Devices
C	3.0h	40.0 marks	Phone emulator / simulator

GPS coordinates for testing the current-location feature

- IFC Tower Loading Bay: 22.285522, 114.158144
- Pacific Place Service Road: 22.277813, 114.166221
- Times Square Cargo Lift: 22.278235, 114.182853
- Harbour City Gate 3: 22.296397, 114.168571
- Olympic City Delivery Point: 22.317847, 114.160072

Reference Wireframes

		
App Icon	Sign-in	Add Cash Stop
		
Daily Run	Check Out Summary	Record List

General Demands

1. Finish the function of this application according to the demands.
2. All text in quotation marks should match the elements given in the brief.
3. The package name / bundle identifier is expected to contain "xx.com.worldskills.cashescort".
4. The application should show the application icon (media-files/media/appIcon.png) on the home screen.
5. All non-authentication main pages should contain a bottom navigation bar with "Daily Run" and "Records".
6. The application should use biometric sign-in by Touch ID / Face ID / device biometric prompt.
7. The application should request location permission when the user uses the current-location input. You can assume the user accepts the request.
8. All data should be stored locally and remain visible after restarting the app.
9. The first page for a fresh start must be the Sign In Page.
10. Submit your work to the provided repository and package the output as instructed.

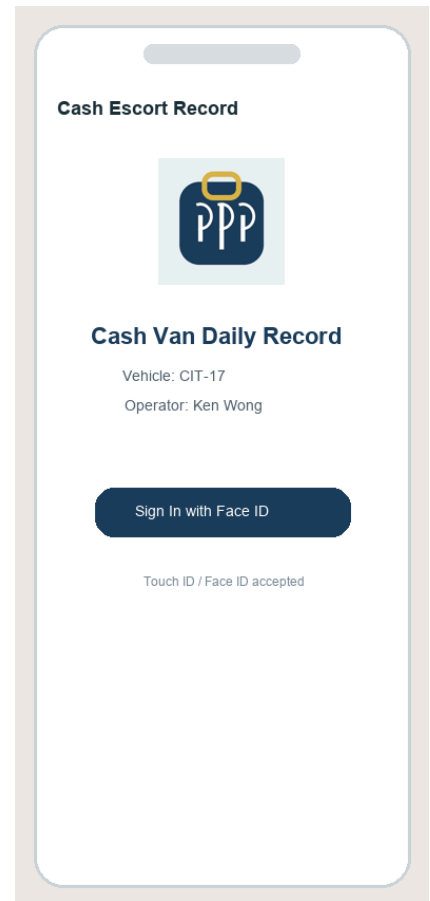
1. Sign In Page

1.1 Elements include:

1. App title label
2. Vehicle / operator information labels
3. Biometric sign-in button
4. Supporting text indicating Touch ID / Face ID sign-in

1.2 Functional requirements:

1. The Sign In Page is the first page on fresh launch.
2. The biometric button should trigger a system biometric prompt.
3. You can assume the device supports Face ID / Touch ID / biometric authentication and the user authenticates successfully.
4. After successful authentication, the app navigates to the Daily Record Page.
5. The sign-in state should remain available (Goes to background and then resumes foreground) until the competitor forces quit the app or the system kills the app process.



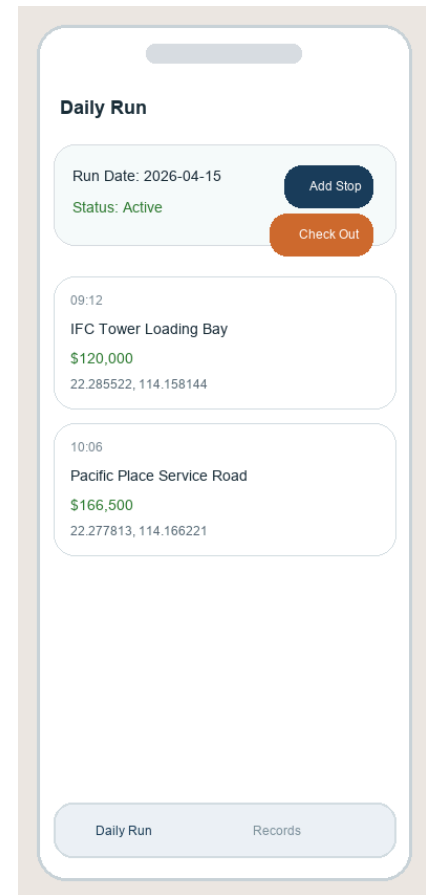
2. Daily Record Page

2.1 Elements include:

1. Navigation Bar with “Daily Run” title
2. Run date label and active status label
3. “Add Stop” button
4. “Check Out” button
5. Stop list showing time, spot name, cash amount, and GPS location
6. Bottom navigation bar with “Daily Run” and “Records”

2.2 Functional requirements:

1. The user can create one active record per calendar day. If a record already exists for the current day, the app should open that record instead of creating a duplicate.
2. The run date should show the selected day of the active record.
3. The stop list should show the saved stops of the active record in ascending input order. And all components are shown correctly.
4. Clicking “Add Stop” should open the Add Cash Stop Page.
5. Clicking “Check Out” should open the Check Out Summary Page.
6. If there are no stops in the active record, the app should design and show a proper empty-state UI.
7. The active record and its stops must remain visible after restarting the app.



3. Add Cash Stop Page

3.1 Elements include:

1. Navigation Bar with “Add Cash Stop” title and “Cancel” / “Save” buttons
2. Cash in amount input
3. Spot name input
4. Latitude input and Longitude input
5. “Use Current Location” button
6. Error message label

3.2 Functional requirements:

1. Clicking “Use Current Location” should request location permission when needed and then fill the latitude and longitude fields.
2. The latitude and longitude fields should still allow manual editing after current location is used.
3. Clicking “Save” validates the fields. If valid, save the stop to the active record with the current time and dismiss the page.
4. The saved stop should immediately appear in the Daily Record Page.
5. Field validation requirements:
 - 5.1 The cash in amount should not be empty and must be numeric and greater than 0.
 - 5.2 The spot name should not be empty.
 - 5.3 The latitude should not be empty and must be numeric.
 - 5.4 The longitude should not be empty and must be numeric.
6. When validation fails, show a suitable error message and do not dismiss the page.
7. Clicking “Cancel” dismisses the page without saving.

Add Cash Stop

Cancel Save

Cash In Amount:

\$120000

Spot Name:

IFC Tower Loading Bay

Latitude:

22.285522

Longitude:

114.158144

Use Current Location

Error message label

4. Check Out Summary Page

4.1 Elements include:

1. Navigation Bar with “Check Out Summary” title
2. Run date label
3. Total cash in amount label
4. Travel distance label
5. Stop count label
6. “Save” and “Cancel” buttons

4.2 Functional requirements:

1. The total cash in amount equals the sum of all cash in amounts in the active record.
2. The travel distance equals the sum of the distances between consecutive stops in input order: stop1 to stop2 + stop2 to stop3 + ... + stop(n-1) to stop(n).
3. If the active record has fewer than two stops, the travel distance should be 0.
4. Clicking “Save” should complete the active record, store the total cash in amount and travel distance, and save the record locally.
5. After save, the record should appear correctly in the Records List Page.
6. Clicking “Cancel” dismisses the page without completing the record.

The screenshot shows a mobile application interface for a 'Check Out Summary' page. At the top, there is a title bar with the text 'Check Out Summary' and a subtitle 'Run Date: 2026-04-15'. Below this, there are three summary cards: 'Total Cash In' with a value of '\$438,500', 'Travel Distance' with a value of '6.42 km', and 'Stops' with a value of '3'. At the bottom of the page, there are two buttons: a prominent orange 'Save' button and a light blue 'Cancel' button.

Category	Value
Total Cash In	\$438,500
Travel Distance	6.42 km
Stops	3

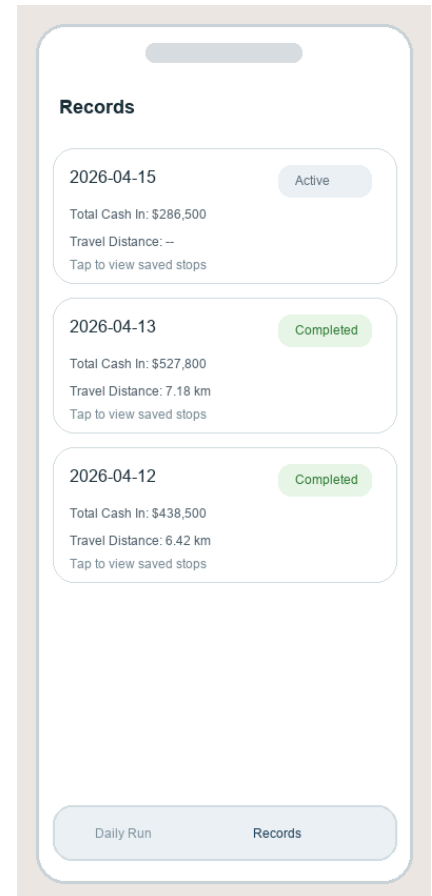
5. Records List Page

5.1 Elements include:

1. Navigation Bar with “Records” title
2. Record list with date, status, total cash in amount, and travel distance
3. Bottom navigation bar with “Daily Run” and “Records”
4. Empty-state UI for no records

5.2 Functional requirements:

1. The list should show all saved records stored locally. And the list should remain correct after restarting the app.
2. Sort the record list in descending order by record date.
3. Completed records should show the saved total cash in amount and saved travel distance.
4. Active records should show the active status properly.



Calculation Rules and Testing Notes

Item	Rule
Total Cash In	Sum of all cash in amounts in the active record.
Travel Distance	Sum of consecutive stop distances in input order: stop1 to stop2 + stop2 to stop3 + ... + stop(n-1) to stop(n).
Single Stop Route	If the record has fewer than two stops, the travel distance is 0.
Saved Result	After check out, store the total cash in amount and the final travel distance locally.

Preloaded data files

- data/operator.json contains the operator and vehicle information.
- data/records.json contains preloaded completed records for testing the record list page.

Instructions to the Competitor

1. Compress the source files, related resources, and exported executable package into XX_Module_C.zip and place it inside the XX_Module_C folder.
2. Export the mobile application according to the target platform requirements and place the executable output in the same folder.
3. Push the entire XX_Module_C folder to the provided remote repository. XX refers to your country code.

Marking Scheme

Judgement: The app is user friendly and well implemented, and the user interface aligns with the given specification.

No.	Sub-criterion	Mark
1	Sign In Page	3.0
2	Daily Record Page	5.2
3	Add Cash Stop Page	5.1
4	Check Out Summary Page	4.4
5	Records List Page	3.0
6	General Demands	2.3
7	Judgement	2.0
	Total	25.0